

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

CORSO DI ARCHITECTURE SOFTWARE DESIGN

PROF. FASOLINO - A.A. 2025 – 26

GRUPPO D5 – CREAZIONE SCALATA

MORRA FRANCESCO DE9000079

NIZZO MATTEO DE9000091

PAESANO GIULIO DE9000080



Sommario

0 - Identificazione del Team e Definizione degli Obiettivi del Progetto	3
1- Analisi dei Requisiti.....	4
2- Progettazione della soluzione	9
3 – Implementazione	15
4 - Test effettuati	23
5 – Sviluppi futuri	31



0 - Identificazione del Team e Definizione degli Obiettivi del Progetto

ID del Team: **D5**

Componenti del Gruppo:

- **Francesco Morra DE9000079** – francesco.morra8@studenti.unina.it
- **Matteo Nizzo DE9000091** – m.nizzo@studenti.unina.it
- **Giulio Paesano DE9000080** – gi.paesano@studenti.unina.it

URL della Versione di Partenza: <https://github.com/Testing-GameSAD-2023/A13>

URL del Repository del Progetto Concluso: <https://github.com/Kekkomo/A13>

Descrizione Sintetica del Task Assegnato

Si vogliono realizzare le funzionalità amministratore di gestione delle “Scalate”.
L’amministratore deve poter accedere tramite interfaccia grafica alle funzioni di:

- Creazione di una Scalata
- Visualizzazione delle Scalate esistenti
- Cancellazione di una Scalata esistente

Una Scalata è composta da un numero variabile di livelli. Per ogni livello, l’Amministratore deve poter selezionare una classe da testare (differente per ogni livello) e scegliere un Robot avversario da sfidare

Approccio di Sviluppo e Gestione del Progetto

Il gruppo ha adottato un approccio Agile ed Iterativo, e a seguito di una prima stesura e prioritizzazione dei requisiti, il processo di sviluppo ha attraversato 3 iterazioni:

- **Iterazione 1:** analisi approfondita dell’architettura preesistente, stima dell’impatto del task e abilitazione delle navigazioni di interesse.
- **Iterazione 2:** implementazione delle funzionalità a maggiore priorità con contemporaneo sviluppo ed integrazione del frontend.
- **Iterazione 3:** implementazione delle funzionalità a minore priorità e irrobustimento del backend.

I tool di sviluppo impiegati sono stati:

- Git, per il version control
- IntelliJ Idea, per il code editing

Il team ha collaborato mediante **riunioni frequenti** di aggiornamento sullo stato di sviluppo e al termine di ogni iterazione per verificare il raggiungimento del goal.

Il team ha applicato il **Pair Programming** per sfruttare efficientemente il tempo a disposizione e scrivere codice di qualità fin da subito

Questo approccio ha permesso di ottenere un’evoluzione graduale e controllata del software, riducendo i rischi e garantendo un miglioramento costante della qualità del prodotto finale.

1- Analisi dei Requisiti

Glossario dei termini

In collaborazione con il **gruppo D1**, responsabile del **task R5**, è stato concordato un **glossario comune dei termini**, con l'obiettivo di favorire l'**interoperabilità** e una comprensione condivisa tra i diversi team di sviluppo.

Termine	Descrizione	Sinonimo
Scalata	Identifica un insieme di livelli	Partita, Gioco
Livello	Identifica un insieme di round	
Round	Identifica una singola sfida contro un avversario su una determinata classe	
Amministratore	Ruolo di gestione del sistema	

Requisiti

ID	Descrizione	Priorità
RF-1	L'amministratore deve poter creare una scalata	1
RF-2	L'amministratore deve poter visualizzare le scalate esistenti	1
RF-3	L'amministratore deve poter eliminare una scalata esistente	2
RF-4	Il microservizio T5 deve poter recuperare le scalate esistenti	3
RF-5	Il microservizio T5 deve poter recuperare i livelli di una scalata esistente	3
RD-1	La scalata è composta da livelli	1
RD-2	Un livello deve tenere traccia della classeUT, Robot da sfidare e il tempoMax	1
V-1	Una scalata deve avere un nome unico	2
V-2	Una scalata deve avere almeno 2 livelli	2
V-3	Un livello deve avere tempoMax come numero non negativo	2
V-4	Una scalata deve ciascun livello con una classeUT diversa	2
V-5	Il sistema deve impedire l'eliminazione di una scalata se esistono giochi avviati ad essa associati	4

Analisi d'impatto

Le funzionalità di base per la gestione della **Scalata** erano già presenti all'interno del microservizio **T1**. Per soddisfare i requisiti del task R4, è stato quindi necessario condurre uno **studio approfondito del microservizio esistente**, al fine di valutare l'impatto delle modifiche e identificare i punti di intervento.

L'analisi ha preso in considerazione i principali **componenti backend**:

- **Scalata.java**: definisce la struttura dell'entità *Scalata*.
- **ScalataRepository.java**: gestisce e astrae la logica di accesso ai dati.
- **ScalataService.java**: implementa la logica di business relativa alle Scalate.
- **ScalataController.java**: espone le funzionalità tramite API REST, mappando le richieste HTTP sui metodi del servizio.
- **ScalataViewController.java**: gestisce le richieste provenienti dalla UI e la risoluzione delle pagine HTML da renderizzare per l'utente.

Oltre al backend, sono stati presi in considerazione anche i **file di frontend**:

- **scalata.js**, **scalata.css** e **scalata.tour**, che gestiscono la logica di interazione lato client e la visualizzazione delle informazioni relative alle Scalate.

Questo studio ha permesso di capire come le nuove funzionalità avrebbero influenzato il microservizio, riducendo i possibili problemi e mantenendo la coerenza con l'architettura esistente.

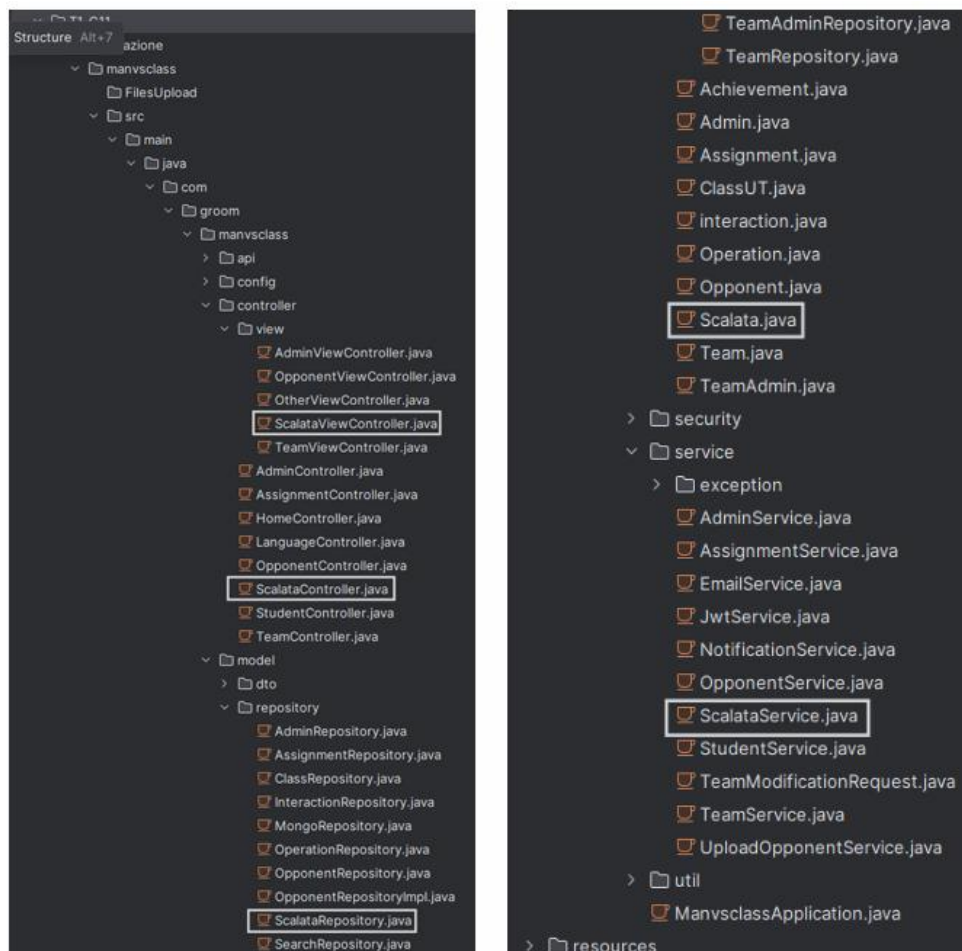
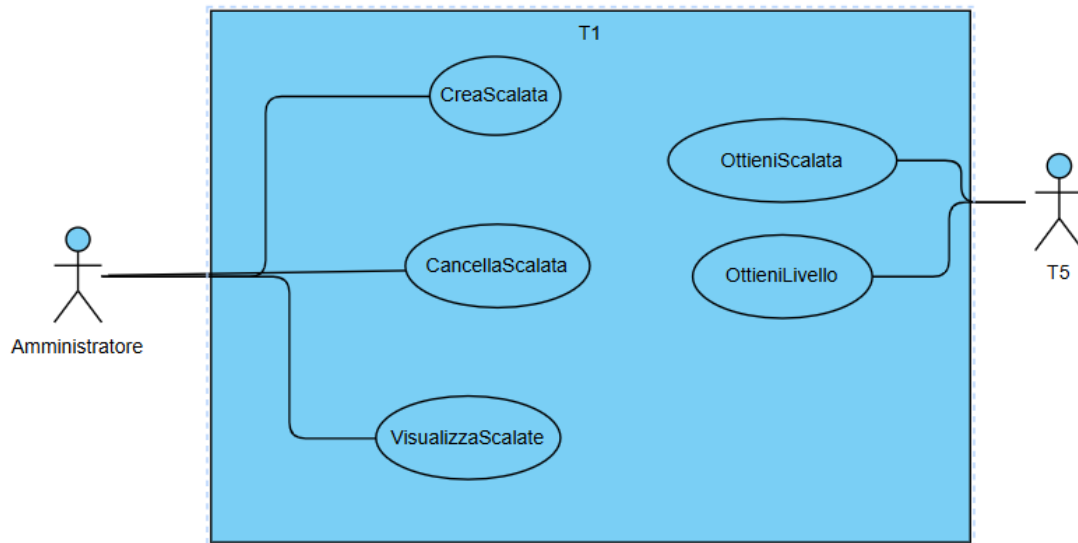


Diagramma dei casi d'uso

Il **diagramma dei casi d'uso** illustra le **principali funzionalità** introdotte dal task R4. I due attori coinvolti sono l'Amministratore, che interagisce direttamente con il sistema tramite la UI, e il microservizio T5, responsabile di richiedere i dati relativi a Scalate e Livelli.



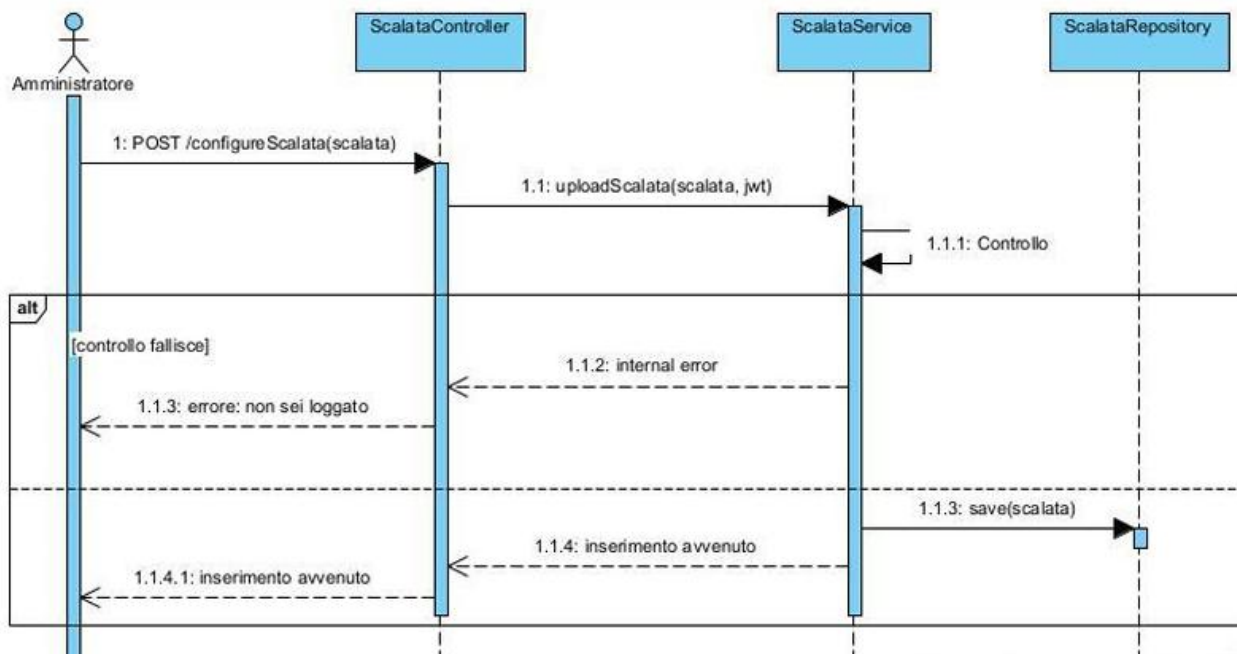
Scenari

Caso d'uso	CreaScalata
Attore primario	Amministratore
Descrizione	Creazione di una nuova scalata nel sistema.
Pre-Condizioni	L'amministratore è loggato nel sistema ed è sulla pagina /scalata/main
Sequenza di eventi	1. L'amministratore clicca sul pulsante "Crea Scalata". 2. Il sistema redireziona l'amministratore sulla pagina /scalata/create. 3. L'amministratore inserisce i dati. 4. L'amministratore clicca sul pulsante "Conferma". 5. Il sistema verifica se sono soddisfatti i seguenti vincoli: 5.1. Non esiste una scalata con uguale nome. 5.2. La scalata è composta da almeno 2 livelli. 5.3. I livelli hanno un tempoMax positivo e non nullo. 5.4. I livelli non hanno la stessa classe da testare. 6. Se soddisfatti: 6.1. viene salvata la scalata nel database 6.2. IL sistema redirezione l'amministratore sulla pagina /scalata/main. 7. Altrimenti: 7.1. viene ritornato ERRORE relativo al vincolo.
Post-Condizioni	Il sistema restituisce una risposta con l'esito dell'operazione.

Caso d'uso	OttieniLivello
Attore primario	T5
Descrizione	Recupero di un Livello di una Scalata.
Pre-Condizioni	Il sistema deve essere in esecuzione e raggiungibile tramite RESTAPI.
Sequenza di eventi	1. Il servizio T5 richiede il Livello fornendo: nome della Scalata e numero del Livello. 2. Il sistema recupera la Scalata attraverso il nome. 3. Se non esiste, ritorna ERRORE. 4. Altrimenti, recupera il Livello dalla Scalata attraverso il numero. 5. Se non esiste, ritorna ERRORE. 6. Altrimenti, il sistema restituisce il livello recuperato.
Post-Condizioni	Il servizio T5 ha ricevuto correttamente l'oggetto JSON contenente l'oggetto.

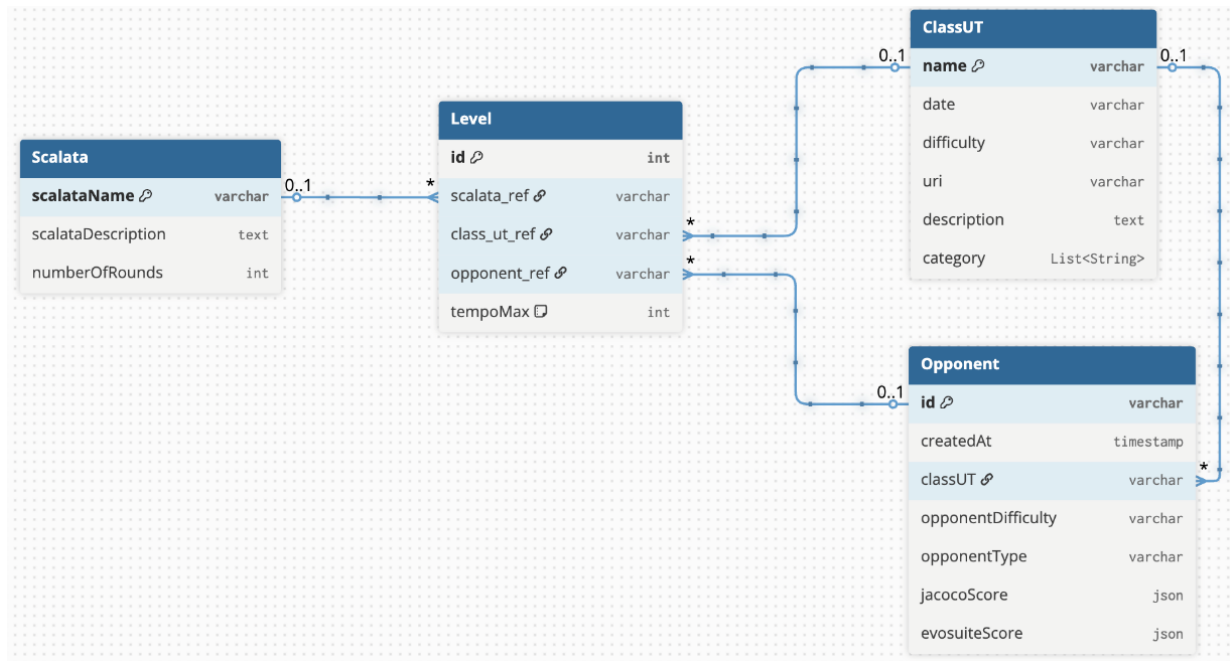
Diagramma di sequenza

Il seguente **diagramma di sequenza** illustra in maniera semplificata il flusso di creazione di una Scalata a livello di backend, senza considerare il frontend. Il diagramma mostra le interazioni tra le componenti principali del microservizio T1, evidenziando come la richiesta dell'Amministratore venga gestita dal controller, elaborata dal service, e infine persistita tramite il repository, includendo la gestione dei dati relativi a Scalata e Livelli.



Modellazione dei dati

Il seguente diagramma è un **diagramma delle classi ad alto livello**, concordato con il gruppo che sviluppa il **game engine** della Scalata, per evidenziare il concetto di **livello** e mostrare come le nuove entità si inseriscono nell'architettura esistente



2- Progettazione della soluzione

Decisioni di progetto per il soddisfacimento dei requisiti

Per soddisfare i requisiti assegnati dal task R4, le modifiche sono state concentrate sul microservizio **T1**. In particolare, le **decisioni** di progetto **principali** sono state:

1. **Modello dei dati:** il modello è stato aggiornato introducendo il concetto di **livello**, in modo da rappresentare correttamente la struttura di una Scalata e le relazioni tra le entità coinvolte. **Il livello non è stato modellato come una collection separata**, poiché utilizzando un database **MongoDB** si è preferito evitare ridondanza: i livelli sono contestuali esclusivamente alla Scalata e saranno memorizzati direttamente nella collection *Scalata*.
2. **Rotte REST API:** sono state realizzate nuove API che consentono sia all'**Amministratore** di accedere alle funzionalità di creazione, visualizzazione e cancellazione delle Scalate, sia al microservizio **T5** di recuperare i dati relativi a Scalate e Livelli da utilizzare all'interno del game engine.

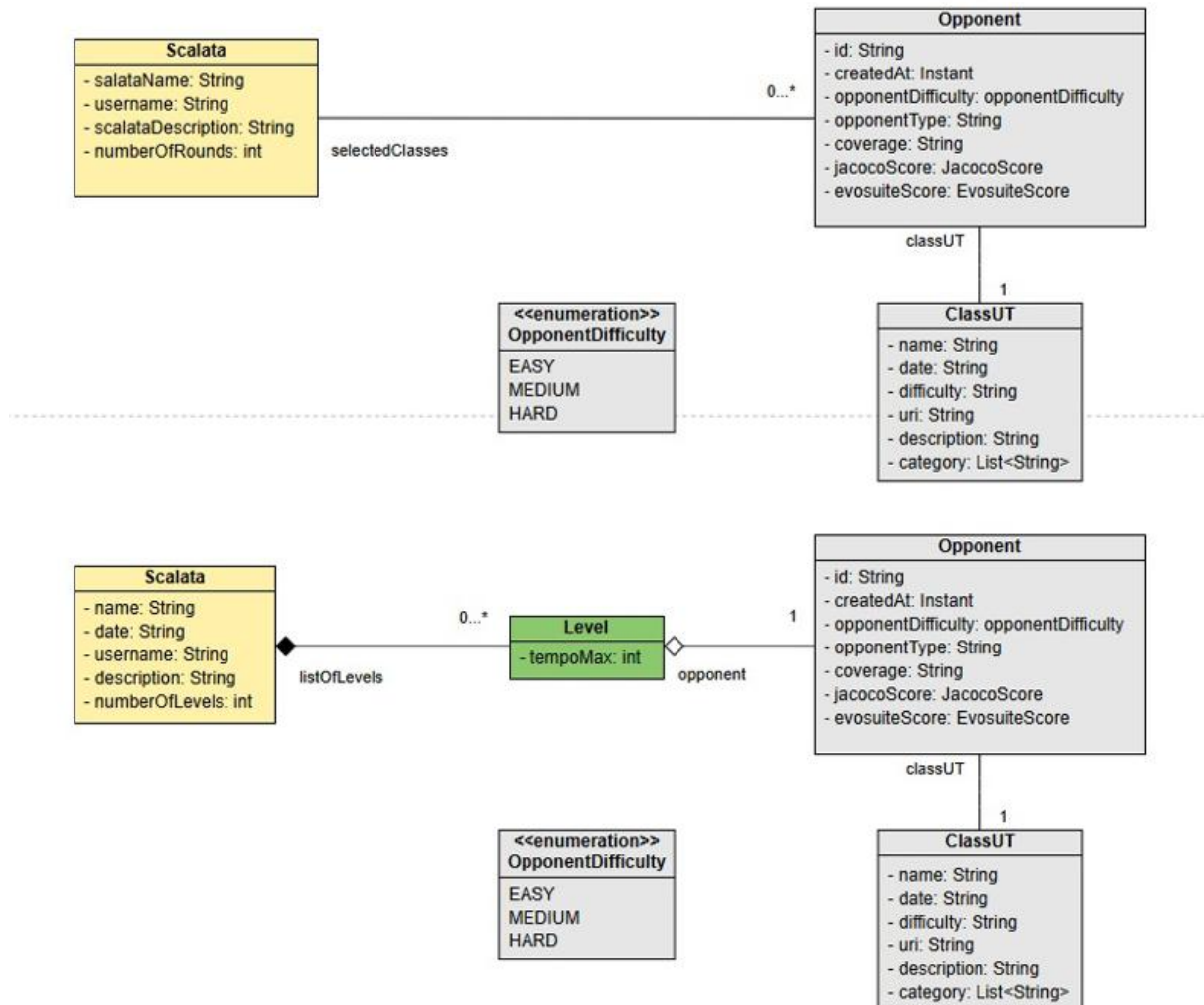
Queste scelte hanno permesso di rispettare i requisiti funzionali, mantenendo la coerenza con l'architettura esistente del microservizio T1 e garantendo l'interoperabilità tra i componenti del sistema.

Diagramma delle classi di progettazione

Il seguente **class diagram** rappresenta la struttura principale del microservizio **T1** aggiornata per il soddisfacimento dei requisiti del task R4. Il diagramma mostra le principali entità coinvolte nella gestione delle **Scalate**, le loro relazioni.

In particolare, viene evidenziato il concetto di **Livello** con annesso tempoMax, memorizzato all'interno della collection *Scalata* nel database MongoDB. Non esiste una relazione diretta tra **Livello** e **ClasseUT**, poiché l'entità **Opponent** già contiene sia il **Robot** sia la **ClasseUT**: tramite **Opponent**, il livello può recuperare la classe senza introdurre ridondanza.

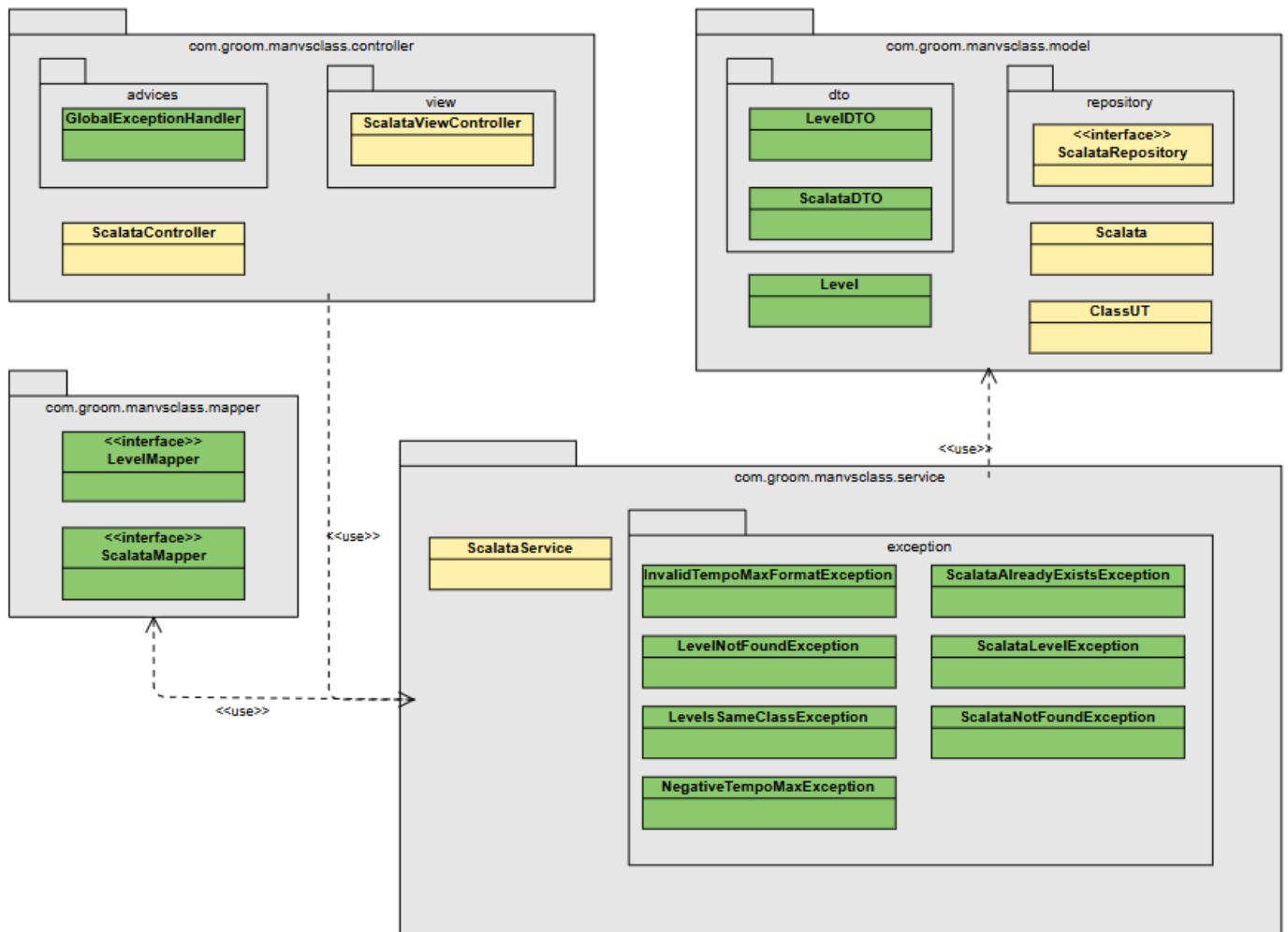
Di seguito è riportato il diagramma delle classi prima e dopo l'aggiunta di Level.



Legenda:

- Nuovo
- Modificato
- Immutato

Diagramma dei Package



Legenda:

- Nuovo
- Modificato
- Immutato

REST API

Sono state introdotte **nuove REST API** nel **microservizio T1**, anche se API simili erano già presenti. La scelta di rifarle è stata dettata dalla necessità di uniformarle e garantire coerenza tra tutte le operazioni disponibili.

Le **API** permettono all'**Amministratore** di **creare, visualizzare e cancellare Scalate** e al **microservizio T5** di **recuperare i dati di Scalate e Livelli**, assicurando **interoperabilità** tra i componenti e riduzione della ridondanza dei dati. Gli esempi di utilizzo sono **documentati** tramite **SpringDoc**.

POST	/scalata/create	Create a new ladder	▼
GET	/scalata/getAll	Retrieve all available ladders	▼
GET	/scalata/get/{name}	Retrieve a ladder by name	▼
GET	/scalata/getLevel/{name}/{number}	Retrieve a specific level of a ladder	▼
GET	/scalata/getLevels/{name}	Retrieve the levels of a ladder	▼
DELETE	/scalata/delete/{name}	Delete a ladder by name	▼

Esempi di REST API del microservizio T1

1. Creazione di una Scalata

- **Metodo:** POST
- **Endpoint:** /scalata/create
- **Body:** ScalataDTO (nome, descrizione, numero livelli, lista livelli)
- **Risposte possibili:**
 - **200 OK** – Scalata creata con successo
 - **400 Bad Request** – Dati non validi
 - **409 Conflict** – Scalata già esistente

2. Recupero di un singolo livello di una Scalata

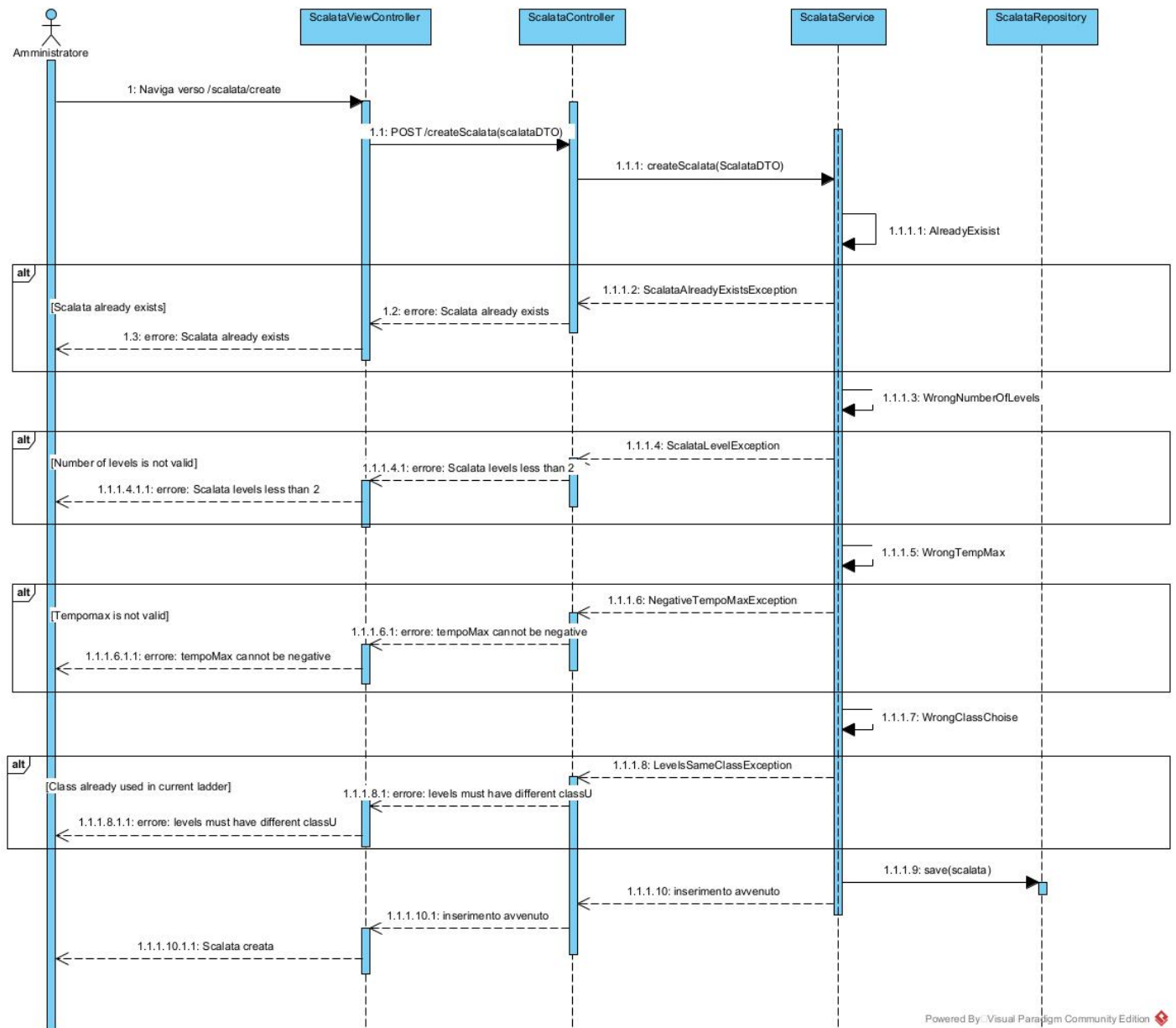
- **Metodo:** GET
- **Endpoint:** /scalata/getLevel/{name}/{number}
- **Risposta:** LevelDTO del livello richiesto
- **Codici possibili:**
 - **200 OK** – Livello trovato
 - **404 Not Found** – Scalata o livello non trovato

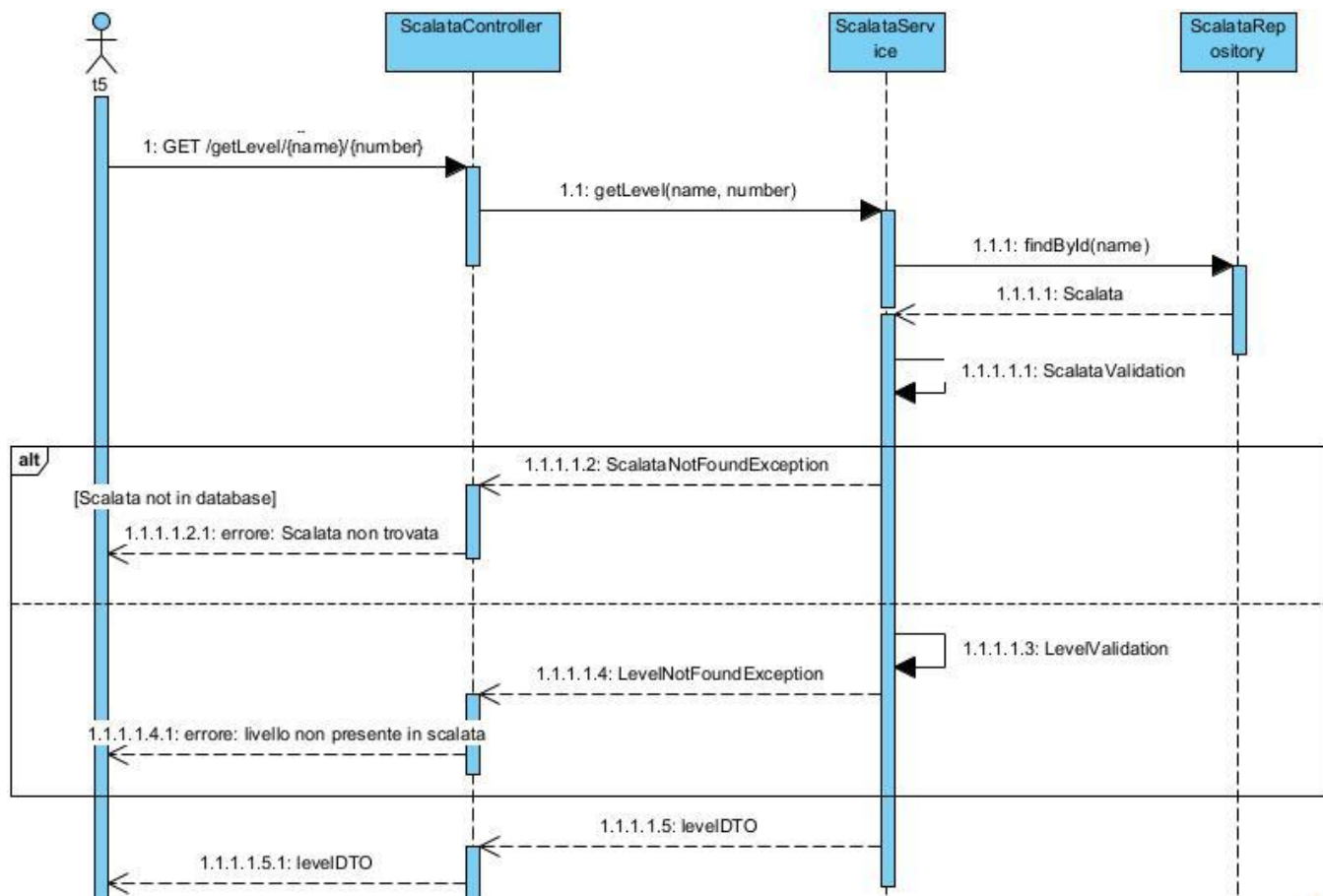
3. Cancellazione di una Scalata

- **Metodo:** DELETE
- **Endpoint:** /scalata/delete/{name}
- **Risposte possibili:**
 - **200 OK** – Scalata eliminata
 - **404 Not Found** – Scalata non trovata

Diagrammi di sequenza

I diagrammi di sequenza riportati illustrano in dettaglio le interazioni tra i componenti del microservizio **T1** per due operazioni fondamentali: la **creazione di una Scalata** e il **recupero di un livello specifico**





3 – Implementazione

Refactoring effettuato

Abbiamo analizzato il codice esistente per valutarne evolvibilità e manutenibilità, individuando quattro criticità principali che ne compromettono affidabilità e scalabilità

- **Assenza del Pattern DTO:** Mancanza completa dell'utilizzo del pattern DTO, essenziale per garantire una separazione netta tra il modello del dominio e i dati scambiati tra i livelli del sistema o nelle chiamate API.
- **Mancanza di Gestione delle Eccezioni:** Assenza totale di meccanismi di gestione delle eccezioni all'interno della logica del backend. Questo rende il sistema vulnerabile a crash in caso di errori operativi o di runtime, impedendo una risposta coerente e sicura al client.
- **Chiamate API Dirette:** Le chiamate alle API vengono effettuate in modo diretto, senza transitare attraverso un API Gateway. Questa scelta preclude i benefici di sicurezza, monitoraggio, bilanciamento del carico e gestione centralizzata dei servizi offerti da un Gateway.
- **Problematiche nell'Interfacciamento con il Database:** Sono state riscontrate svariate criticità nell'interfacciamento con il database, che impediscono la corretta e affidabile interazione con lo strato di persistenza dei dati.

Per risolvere le **problematiche** identificate, sono state **introdotte** le classi **ScalataDTO** e **LevelDTO** per garantire la separazione tra il modello di dominio e i dati scambiati tra i livelli del sistema e nelle chiamate API. La gestione delle eccezioni è stata centralizzata tramite un **Exception Handler globale**, in modo da intercettare eventuali errori e fornire risposte coerenti e sicure ai client intercetta diversi tipi di errori specifici della logica delle Scalate:

- **ScalataLevelException:** errore generico durante la definizione dei livelli.
- **LevelSameClassException:** errore di validazione quando due livelli della stessa Scalata contengono la stessa classe utente.
- **ScalataNotFoundException:** la Scalata richiesta non esiste.
- **ScalataAlreadyExistsException:** tentativo di creare una Scalata già esistente.
- **NegativeTempoMaxException:** valore di tempo massimo negativo non consentito.

Sul **fronte client-side**, il JavaScript **scalata.js** **combinava** in modo **improprio presentazione e logica**, generando **complessità** e **scarsa manutenibilità**. È stato quindi deciso di effettuare i seguenti interventi:

- **Rimozione del vecchio scalata.js:** eliminato perché combinava presentation logic e business logic, generando complessità e scarsa manutenibilità.
- **Nuova pagina principale scalata_main.html:** integra **Thymeleaf** per consegnare pagine già renderizzate al client, riducendo la complessità del frontend e aumentando la coerenza architetturale.
- **Pagina di creazione scalata_edit.html:** continua a utilizzare JavaScript per funzionalità dinamiche, come la modifica in tempo reale del numero di livelli.

- **Internazionalizzazione:** l'interfaccia è predisposta per supportare più lingue senza modifiche al codice

Di seguito è **riportata** una **tabella** con tutti i **file modificati**:

File	LOC (-/+)	Descrizione
T1-G11\applicazione\manvsclass		
pom.xml	0 / 23	Aggiunte le dipendenze springdoc (per la documentazione) e mapstruct (per generare i mapper DTO).
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\controller		
ScalataController.java	20 / 123	Ridefinizione dei metodi su cui sono mappate le rotte /scalata.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\controller\advices		
GlobalExceptionHandler.java	0 / 72	Introdotta la classe per la gestione centralizzata delle eccezioni per separare gli interessi rispetto ai controller.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\controller\view		
ScalataViewController.java	7 / 49	Ridefinizione dei metodi su cui sono mappate le rotte frontend /scalata.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\mapper		
LevelMapper.java	0 / 15	Introdotte le classi Mapper per rispettare il pattern DTO.
ScalataMapper.java	0 / 14	
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\model		
ClassUT.java	15 / 3	Snellita la classe mediante libreria lombok.
Level.java	0 / 18	Introdotta l'entità Level.
Scalata.java	74 / 12	Riadattata l'entità Scalata e snellita mediante lombok.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\model\dto		
LevelDTO.java	0 / 15	Introdotte le classi DTO.
ScalataDTO.java	0 / 19	
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\model\repository		
ScalataRepository.java	8 / 8	Modificati i nomi dei metodi per refactoring.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\service		
ScalataService.java	41 / 126	Ridefiniti i metodi che implementano la logica di gestione delle scalate.
T1-G11\applicazione\manvsclass\src\main\java\com\groom\manvsclass\service\exception		
InvalidTempoMaxFormatException.java	0 / 7	Introdotte le eccezioni per aumentare la robustezza.
LevelNotFoundException.java	0 / 7	
LevelsSameClassException.java	0 / 7	
NegativeTempoMaxException.java	0 / 7	
ScalataAlreadyExistsException.java	0 / 7	
ScalataLevelException.java	0 / 7	
ScalataNotFoundException.java	0 / 7	
T1-G11\applicazione\manvsclass\src\main\resources\lang		
messages.properties	0 / 47	Esteso il supporto all'internazionalizzazione.
messages_en.properties	1 / 49	
messages_it.properties	1 / 49	

T1-G11\application\manvsclass\src\main\resources\static\t1\css		
<i>scalata.css</i>	135 / 151	Aggiunte animazioni ai button.
T1-G11\application\manvsclass\src\main\resources\static\t1\js		
<i>dashboard.js</i>	1 / 1	Aggiunto il query parametr <i>tour=true</i> per richiedere la presentazione del tour quando si naviga verso /scalata/main.
<i>scalata.js</i>	600 / 0	Eliminato per riprogettazione del frontend.
<i>scalata_tour.js</i>	180 / 0	Eliminato per riprogettazione del frontend.
T1-G11\application\manvsclass\src\main\resources\static\t1\js\scalata		
<i>scalata_edit.js</i>	0 / 140	Introdotta per gestire la generazione dinamica dei livelli in fase di creazione della scalata, recuperare gli opponents disponibili da /opponents e inviare la nuova scalata su /scalata/create.
<i>scalata_main.js</i>	0 / 14	Introdotta per invocare la rotta /scalata/delete per cancellare una data scalata visualizzata.
<i>scalata_main_tour.js</i>	0 / 192	Riadattato il tour precedente ai nuovi elementi html.
T1-G11\application\manvsclass\src\main\resources\static\t1\js\utils		
<i>api.js</i>	0 / 48	Aggiunte le chiamate wrapper per le nuove rotte.
<i>endpoints.js</i>	0 / 10	Aggiunte le costanti per le nuove rotte.
T1-G11\application\manvsclass\src\main\resources\templates		
<i>dashboard.html</i>	1 / 1	Eliminato attributo <i>disabled</i> al button che naviga verso /scalata/main
T1-G11\application\manvsclass\src\main\resources\templates\scalata		
<i>scalata_edit.html</i>	0 / 97	Aggiunta la pagina per la creazione della scalata.
<i>scalata_main.html</i>	135 / 149	Riprogettata la pagina di visualizzazione delle scalate.
ui_gateway\routes		
<i>T1_route.conf</i>	1 / 1	Aggiunta la navigabilità sulla rotta /scalata/create.

Legenda:

- Nuovo
- Modificato
- Immutato
- Cancellato

Flusso di esecuzione “creazione scalata”

Dopo essersi loggato con il suo account, l'amministratore naviga sulla rotta `/scalata/create` gestita dallo `ScalataViewController` che restituisce la pagina `scalata_edit.html`:

```
@GetMapping("/{create}") @ Matteo Nizzo
public ModelAndView createScalata(@CookieValue(name = "jwt", required = false) String jwt) {
    ModelAndView mv = new ModelAndView(viewName: "scalata/scalata_edit");
    return mv;
}
```

L'amministratore indica i dati della scalata da creare con gli opportuni livelli ad essa associati:

Creazione Scalata			
Nome scalata			
Irraggiungibile			
Descrizione			
La scalata impossibile da sconfiggere			
Editor livelli			
Numero livelli: 2			
Livello 1	CalcolatricevsRandoop(EASY)	Tempo(min) 30	-
Livello 2	FTPFilesEvoSuite(HARD)	Tempo(min) 60	-
		+	
X Annulla		Conferma ✓	

Il pulsante **conferma** invoca una **POST** request sulla rotta `/scalata/create`, che viene gestita dallo `ScalataController`:

```
@Operation( Matteo Nizzo
    summary = "Create a new ladder",
    description = "Checks for duplicates, minimum number of levels, maximum time, and different UT classes")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Ladder created successfully",
        content = @Content(schema = @Schema(implementation = String.class))),
    @ApiResponse(responseCode = "400", description = "Invalid levels or negative maximum time",
        content = @Content(schema = @Schema(implementation = String.class))),
    @ApiResponse(responseCode = "409", description = "Ladder already exists",
        content = @Content(schema = @Schema(implementation = String.class)))
})
@PostMapping("/{create}")
public ResponseEntity<String> createScalata(@RequestBody ScalataDTO scalataDTO,
    @CookieValue(name = "jwt", required = false) String jwt) {
    scalataService.createScalata(scalataDTO);
    return ResponseEntity.ok(body: "Ladder created");
}
```

L'oggetto passato è uno **ScalataDTO** che permette di definire un sottoinsieme dei dati scambiati tra frontend e backend, rispetto a quanti definito nelle entità.

Viene invocato lo *ScalataService* per gestire la **logica di business** che riguarda le entità scalata:

```
public void createScalata(ScalataDTO scalataDTO) { 1 usage  ⤵ Matteo Nizzo +1
    logger.info("[CREATE] Received ScalataDTO: {}", scalataDTO);

    // Riferimento delle classi scelte
    Set<String> classUT = new HashSet<>();

    // Scalata con ugual nome già esistente
    if (scalataRepository.findById(scalataDTO.getName()).isPresent()) {
        throw new ScalataAlreadyExistsException("Scalata already exists");
    }

    // Scalata con meno di 2 livelli
    if (scalataDTO.getNumberOfLevels() < 2) {
        throw new ScalataLevelException("Scalata levels less than 2");
    }

    for (LevelDTO levelDTO : scalataDTO.getListOfLevels()) {
        // Scalata con livelli a tempo negativo o nullo
        if (levelDTO.getTempoMax() <= 0) {
            throw new NegativeTempoMaxException("tempoMax cannot be negative");
        }

        // Scalata con livelli con stessa classe UT
        if (classUT.contains(levelDTO.getOpponent().getClassUT())) {
            throw new LevelsSameClassException("levels must have different classUT");
        } else {
            classUT.add(levelDTO.getOpponent().getClassUT());
        }
    }

    Scalata scalata = scalataMapper.scalatafromScalataDTO(scalataDTO);
    scalata.setName(scalataDTO.getName().trim());
    scalataRepository.save(scalata);

    logger.info("[CREATE] Scalata '{}' saved successfully", scalata.getName());
}
```

Dopo aver verificato che l'istanza di scalata passata soddisfa i vincoli predisposti, si converte l'oggetto **ScalataDTO** in **Scalata** attraverso lo *ScalataMapper*.

L'entità che definisce gli attributi di una scalata è definita in *Scalata*:

- **@Document**
Salva le istanze come documenti nella collection "scalate".
- **@Id**
Usa il nome della scalata come id mongo del documento.

```
@Document(collection = "scalate") 12 usages
@Getter
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class Scalata {

    @Id
    private String name;

    // meta dati creazione
    private String date;
    private String username;

    private String description;
    private int numberOfLevels;
    private List<Level> listOfLevels;

}
```

Poiché i livelli sono contestuali ad una unica scalata, si è scelto di non creare una collection separata.

L'entità che definisce gli attributi di un livello è definita in *Level*:

- **@DBRef**
Mantiene il riferimento ad un documento istanza di *Opponent* salvato nella collection *opponents* attraverso il suo id.

Questo per implementare il contenimento lasco e avere tutte le informazioni dell'Opponent accessibili senza ulteriori richieste GET.

```
@Getter 5 usages Matteo Nizzo
@Setter
@ToString
@NoArgsConstructor
@AllArgsConstructor
public class Level {

    @DBRef
    private Opponent opponent;
    private int tempoMax;

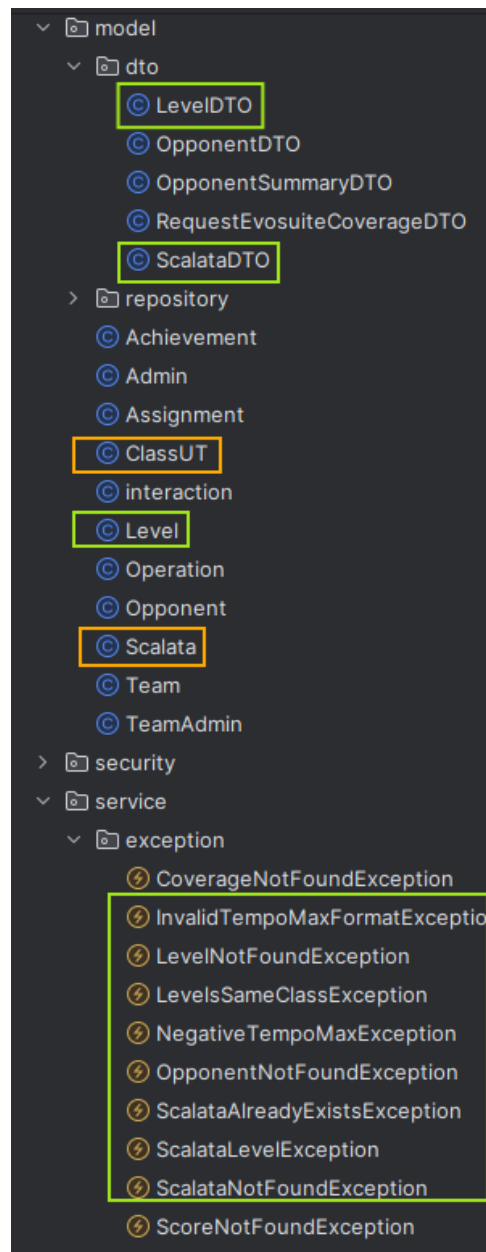
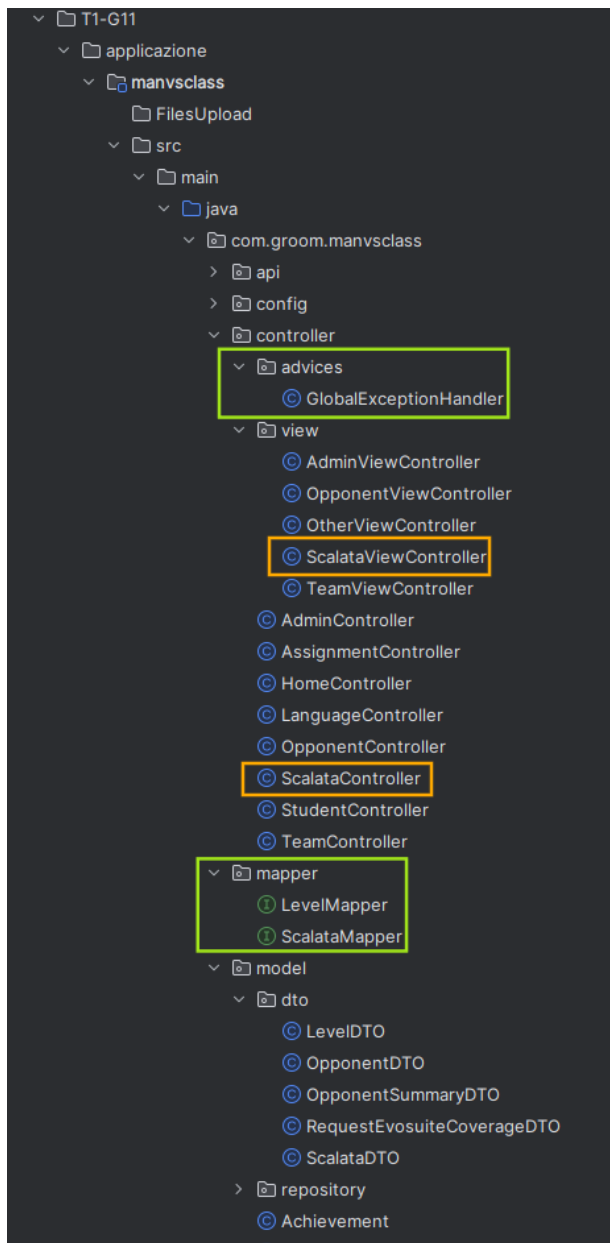
}
```

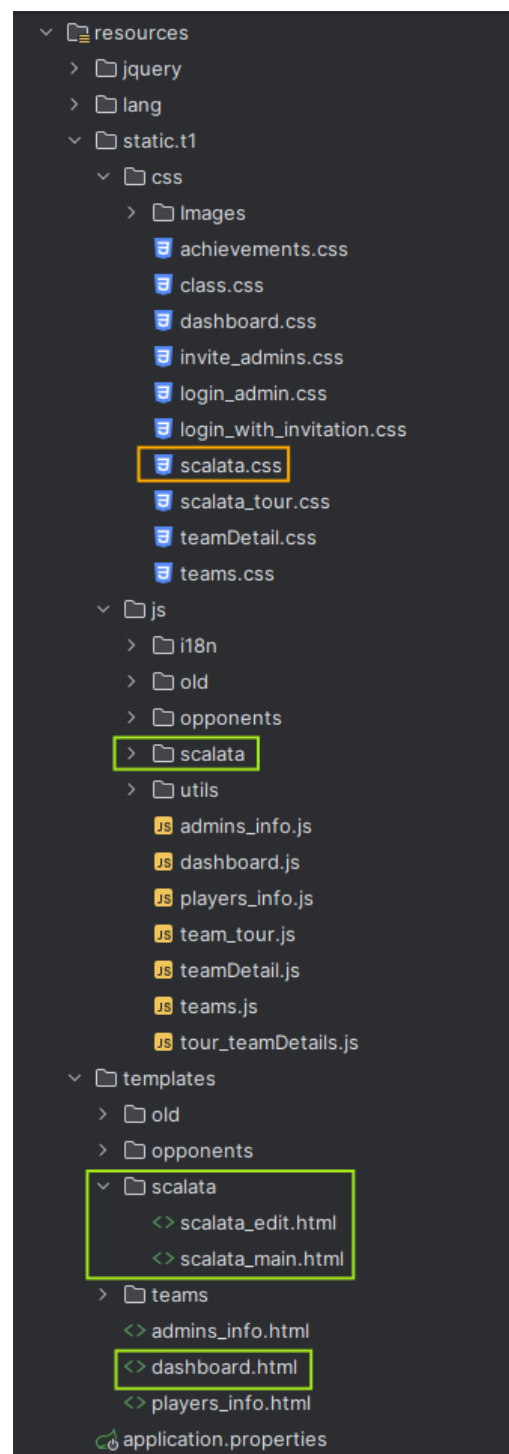
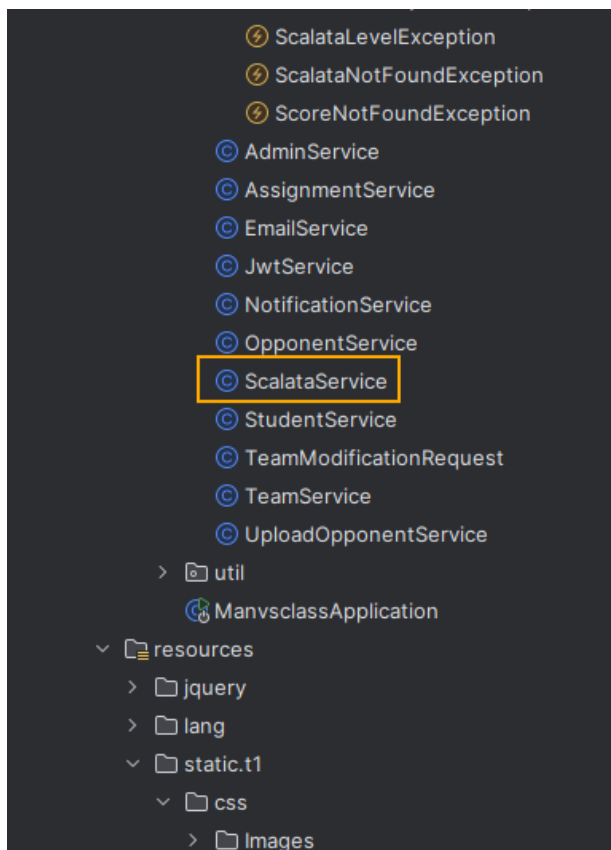
L'istanza di esempio viene salvata sul **MongoDB** come segue:

```
{
  _id: 'Irraggiungibile',
  date: '17/12/25',
  username: 'm.nizzo@studenti.unina.it',
  description: 'La scalata impossibile da sconfiggere',
  numberOfLevels: 2,
  listOfLevels: [
    {
      opponent: DBRef("opponents", ObjectId("692ae3ef7ecf803dcd33bf6d")),
      tempoMax: 30
    },
    {
      opponent: DBRef("opponents", ObjectId("6935bc493b78a068ee63f349")),
      tempoMax: 60
    }
  ],
  _class: 'com.groom.manvsclass.model.Scalata'
}
```

Impatto sulla struttura del progetto

A seguito dell'intervento sul codice del **microservizio T1**, la struttura del progetto è ora la seguente:



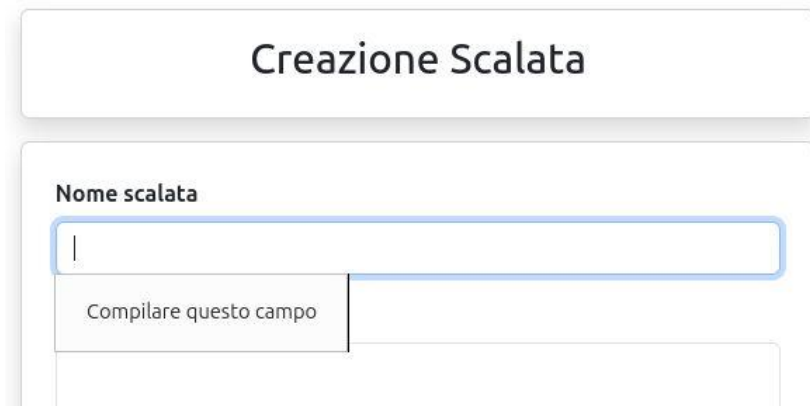


4 - Test effettuati

Test Case ID	Descrizione	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese	Esito
1	Inserimento campo 'Nome scalata' vuoto.	L'amministratore ha selezionato la funzionalità 'crea scalata'.	Nome scalata: "	Avviso di riempimento campo 'Nome scalata'.	La scalata non viene salvata all'interno del database.	PASS
2	Inserimento scalata con nome uguale ad una scalata già esistente.	L'amministratore ha selezionato la funzionalità 'crea scalata', nel database deve essere già salvata una scalata avente lo stesso nome.	Nome scalata = Nome scalata già esistente.	Errore: 'la scalata esiste già'.	La scalata non viene salvata all'interno del database.	PASS
3	Inserimento scalata con numero di livelli minore a 2.	L'amministratore ha selezionato la funzionalità 'crea scalata', nel database non deve essere già salvata una scalata con lo stesso nome della scalata che si intende inserire e sia presente almeno un opponent.	Nome scalata: "Scalata1" Descrizione: "prova" Livello1: opponent valido, tempo>0	Errore: "numero di livelli inferiore a 2"	La scalata non viene salvata all'interno del database.	PASS

4	Inserimento scalata con almeno 2 livelli e tempomax <= 0	L'amministratore ha selezionato la funzionalità 'crea scalata', nel database non deve essere già salvata una scalata con lo stesso nome della scalata che si intende inserire e siano presenti almeno due opponent relativi a diverse classi di test.	Nome scalata: "Scalata2" Descrizione: "prova" Livello1: opponent1 valido, Tempo=2 Livello2: opponent2 relativo a classe diversa da opponent1, Tempo=0	Errore sul campo Tempo: "Selezionare un valore superiore o uguale a 1".	La scalata non viene salvata all'interno del database.	PASS
5	Inserimento di una scalata con opponent relativi a due classi uguali.	L'amministratore ha selezionato la funzionalità 'crea scalata', nel database non deve essere già salvata una scalata con lo stesso nome della scalata che si intende inserire e sia presente almeno un opponent.	Nome scalata: "Scalata3" Descrizione: "prova" Livello1: opponent1 valido, Tempo=2 Livello2: opponent2 relativo a stessa classe di opponent1, Tempo =2	Errore: "I livelli devono avere classiUT diverse"	La scalata non viene salvata all'interno del database.	PASS
6	Inserimento di una scalata avente tutti i campi validi.	L'amministratore ha selezionato la funzionalità 'crea scalata', nel	Nome scalata: "Scalata4"	Scalata Creata con successo!	La scalata viene salvata all'interno del database e diventa visibile all'interno dell'elenco.	PASS

		database non sono presenti scalate con lo stesso nome della scalata che si intende inserire.	Descrizione: “prova” Livello1: opponent1 valido, Tempo=2 Livello2: opponent2 relativo a classe diversa da opponent1, Tempo =2			
7	Eliminazione di una scalata.	L'amministratore ha selezionato dal pannello principale la gestione scalata.	Click su pulsante elimina scalata.	Avviso: “Sei sicuro di voler eliminare questa scalata?”, l'utente seleziona OK	La scalata viene eliminata dal database e non viene più visualizzata all'interno dell'elenco	PASS



Creazione Scalata

Nome scalata

Compilare questo campo

Figura 1 Immagine relativa al test ID 1

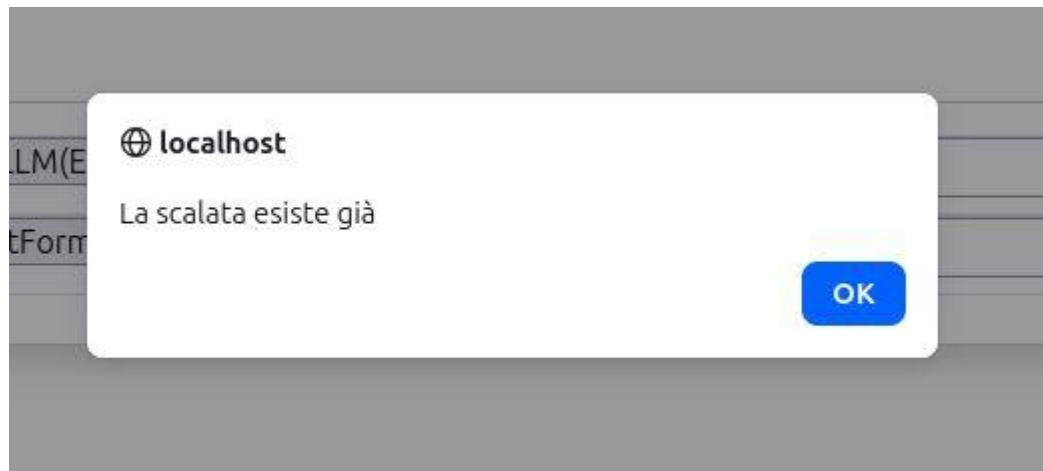


Figura 2 Immagine relativa al test ID 2

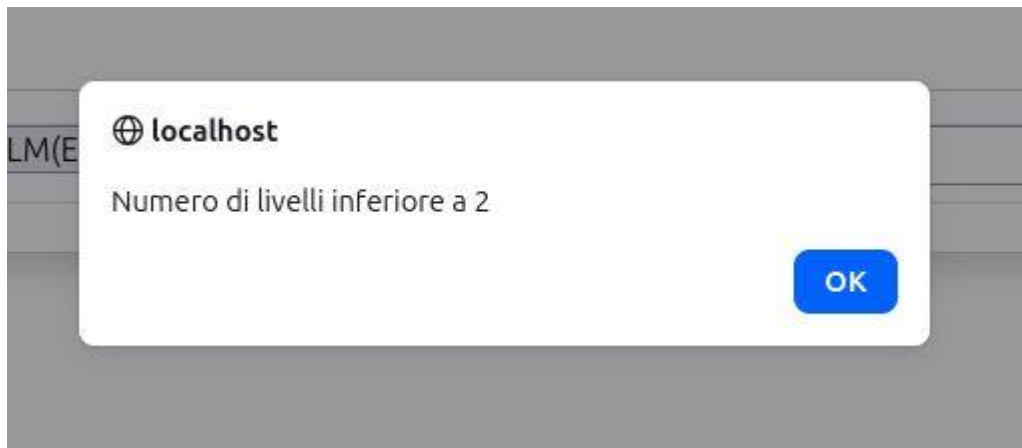


Figura 3 Immagine relativa al test ID 3

Editor livelli +

Numero livelli: 2

Livello 1	FTPvsLLM(EASY) ▼	Tempo(min) 2	–
Livello 2	OutputFormatvsLLM(EASY) ▼	Tempo(min) 0	–

Selezionare un valore superiore o uguale a 2.

✕ Annulla Conferma ▼

Figura 4 Immagine relativa al test ID 4

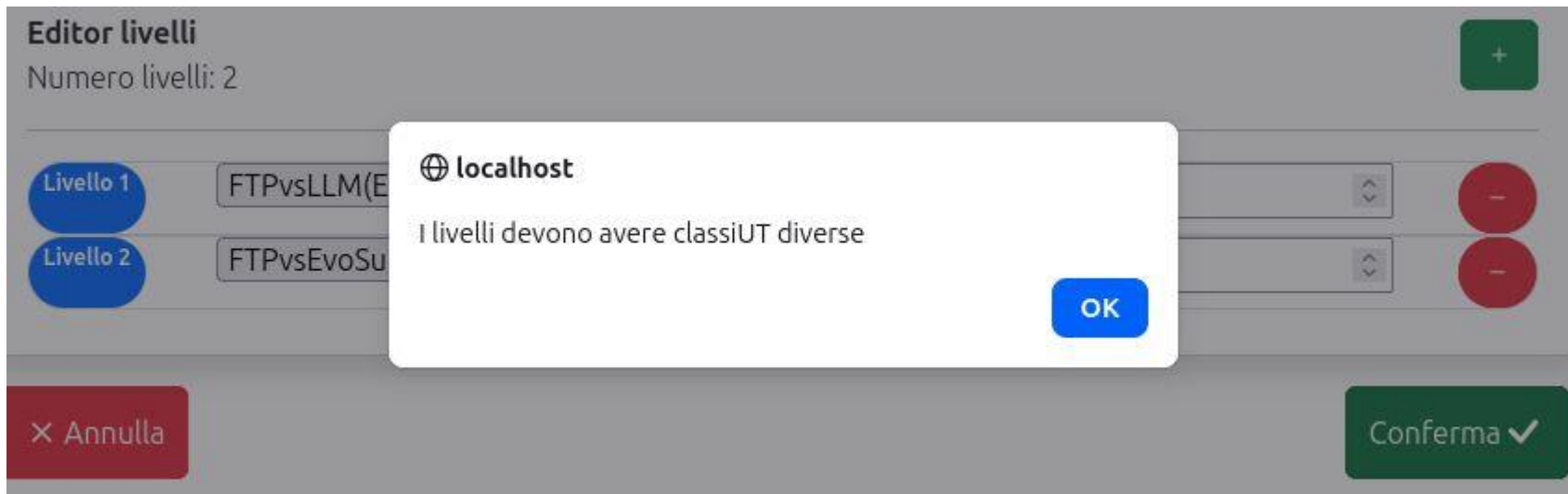


Figura 5 Immagine relativa al test ID 5



Figura 6 Immagine relativa al test ID 6

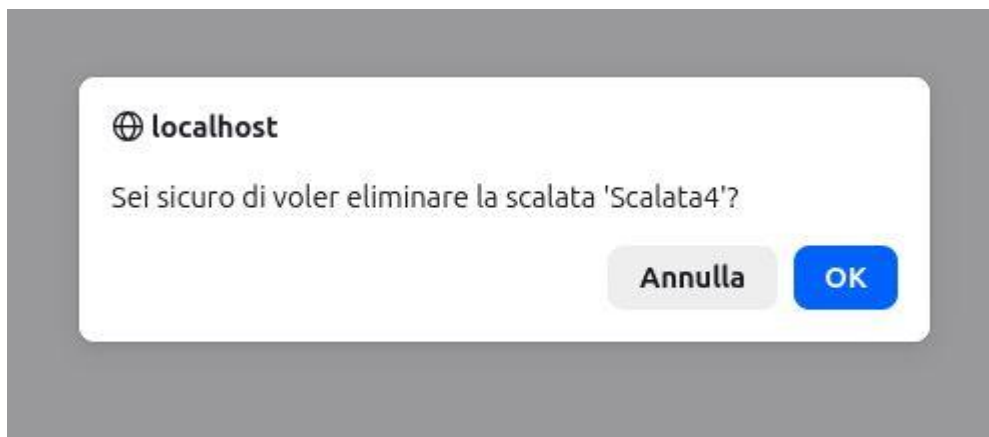


Figura 7 Immagine relativa al test ID 7



Figura 8 Immagine relativa al test ID 7

5 – Sviluppi futuri

- Nella versione attuale del sistema, è stata fatta l'ipotesi che l'amministratore desideri cancellare una "scalata" esclusivamente **immediatamente dopo la sua creazione**, ad esempio per correggere un errore. Di conseguenza, si è partiti dal presupposto che, nel momento in cui l'amministratore avvia la procedura di cancellazione, **nessun utente stia attualmente giocando** quella specifica scalata. Tuttavia, qualora si decidesse di **rimuovere questa ipotesi**, sarebbe necessario definire e implementare il comportamento del sistema nel caso in cui un amministratore tenti di cancellare una scalata che è **correntemente in uso da parte di uno o più utenti**. In ogni caso, l'implementazione di tale comportamento è stato considerato **non opportuno** in quanto avrebbe richiesto l'introduzione di rotte da parte di T5 che finora non erano ancora previste introducendo un **impatto molto significativo ed** entrando anche in **contrasto con l'operato di altri team**.
- La seconda possibile modifica riguarda principalmente un **aspetto tecnico fondamentale del database**. Attualmente, nel campo opponent di Level, viene salvato un **referimento debole**, noto come **DbRef**.
Il problema: Questo approccio implica che se un opponent viene **eliminato**, le relative "scalate" (Level) che lo contengono **non vengono eliminate automaticamente** (manca la **cancellazione a cascata**). Di conseguenza, queste scalate rimarrebbero **visibili** nel sistema, pur **non essendo più giocabili** perché l'avversario associato non esiste più.
La soluzione Tecnologica: Il passaggio a un **database di tipo relazionale** risolverebbe nativamente questa problematica, gestendo automaticamente l'**integrità dei dati** e la **cancellazione a cascata** attraverso l'uso delle **Chiavi Esterne**.